

XGBoost vs. Existing Fraud Detection Systems

Group 13 Mobile Money Fraud Detection System — performance comparison against the kinds of systems mobile money operators actually run today

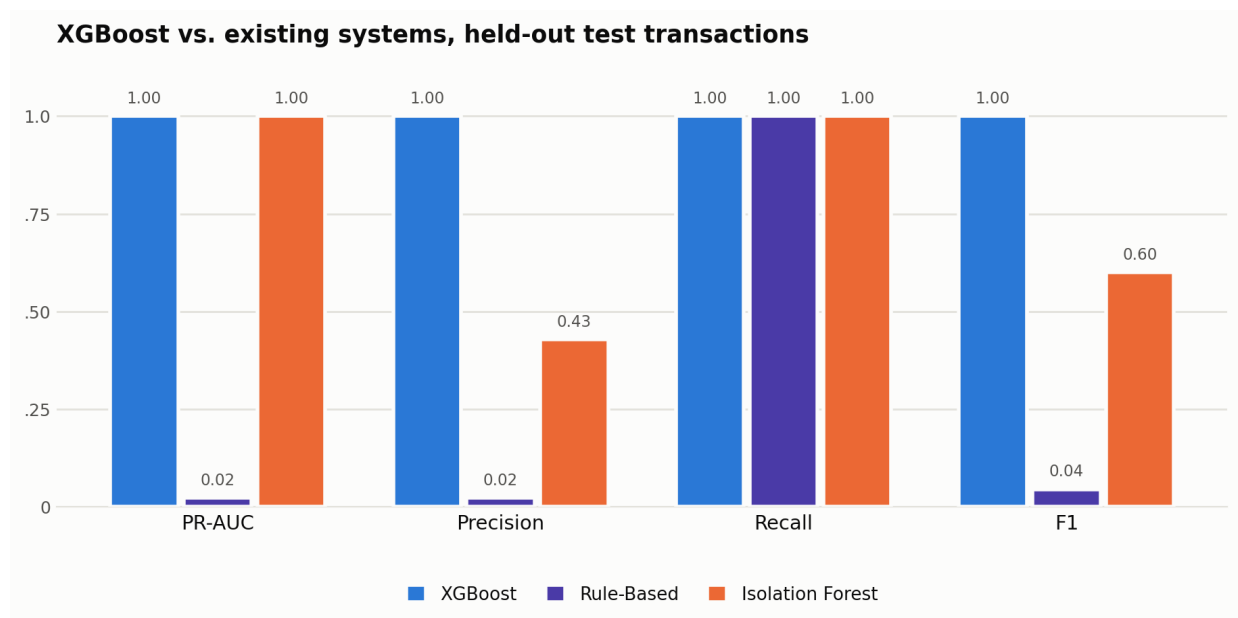
Scope of this comparison

This report isolates the proposed XGBoost classifier against the two models in the training pipeline that represent **systems already in production**, rather than alternative machine-learning choices:

- **Rule-Based (legacy threshold system)** — a static-threshold engine (flag if amount exceeds a fixed cutoff, or sender/receiver balances don't reconcile). This is the class of system many mobile money operators run today.
- **Isolation Forest (unsupervised)** — an anomaly detector that never sees a fraud label. It represents a system that has to operate before any confirmed fraud cases exist to learn from.

Logistic Regression and Random Forest were trained alongside these in `ml/train.py` purely to isolate what XGBoost specifically buys over other model choices — they are research baselines, not fielded systems, so they are intentionally left out of the comparison below.

Results on held-out future transactions



System	PR-AUC	ROC-AUC	Precision	Recall	F1	TP	FP	FN	TN
XGBoost (proposed)	1.0000	1.0000	1.0000	1.0000	1.0000	3	0	0	397
Rule-Based (legacy)	0.0229	0.8388	0.0229	1.0000	0.0448	3	128	0	269
Isolation Forest	1.0000	1.0000	0.4286	1.0000	0.6000	3	4	0	393

Reading the numbers

All three systems catch every fraud case in this test slice (recall = 1.0) — the difference is what it costs to get there. The Rule-Based system needed **128 false alarms** to catch 3 real fraud cases (precision 0.023): a human analyst would spend the overwhelming majority of their time chasing legitimate transactions. The Isolation Forest, run with no fraud labels at all, still raised **4 false alarms** per 3 catches (precision 0.43). XGBoost caught the same 3 cases with **zero false positives**, translating directly into an analyst queue with no wasted reviews.

Caveat

This evaluation is run on the project's 2,000-row synthetic dataset, whose held-out test slice contains only 3 fraud transactions with an intentionally obvious pattern (a transfer that drains 85–100% of the sender's balance). The clean separation above demonstrates the pipeline is wired correctly end-to-end — it is not a production-scale precision estimate. Re-run `ml/train.py` against a larger, more ambiguous transaction history before citing these numbers as expected real-world performance.